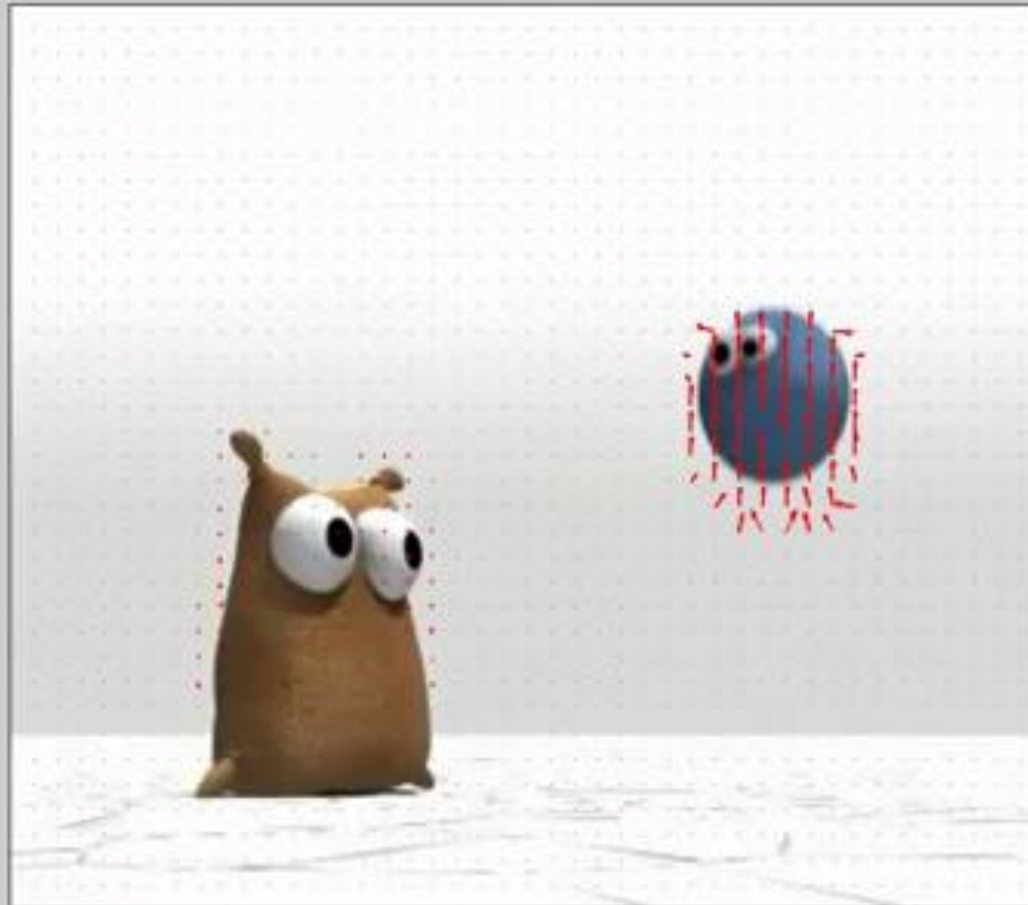


Motion and Optical Flow

Computer Vision: AI3619



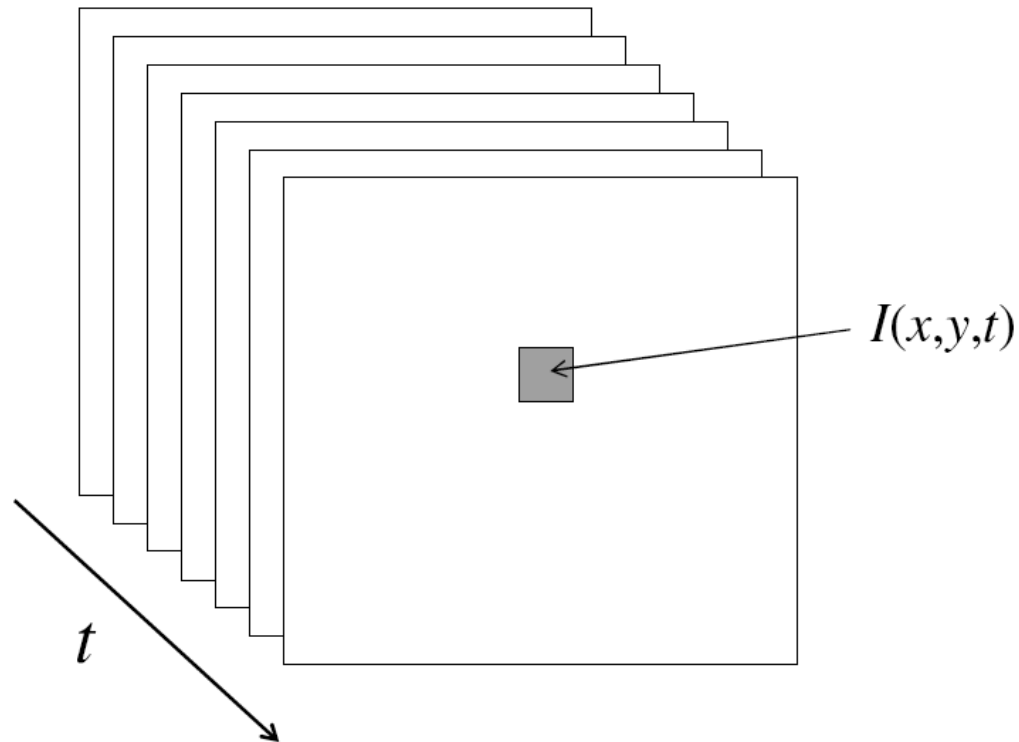
Motion Vectors Frame 2





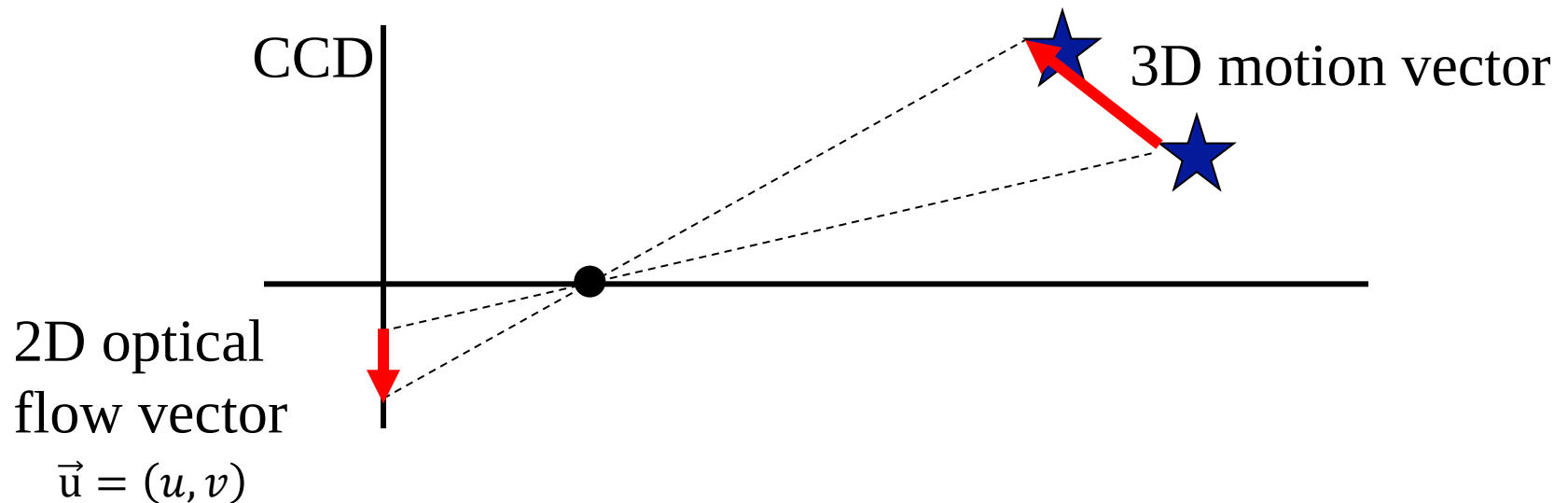
Video

- A video is a sequence of frames captured over time
- Now our image data is a function of space (x, y) and time (t)



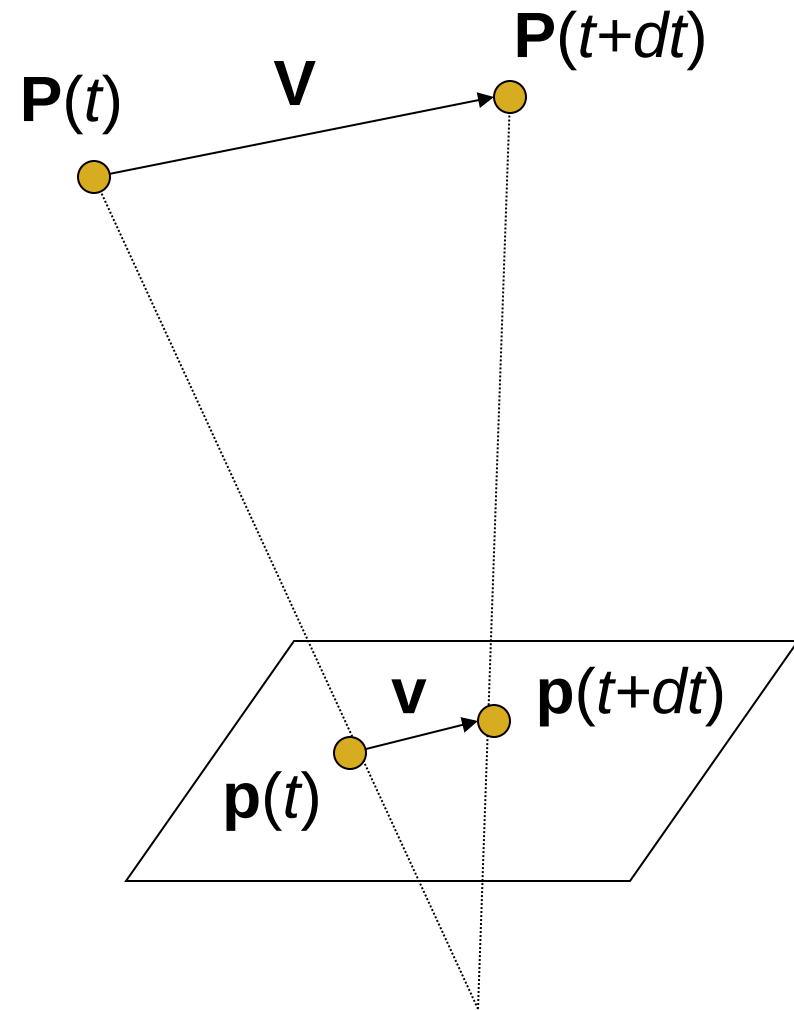
Motion Field & Optical Flow Field

- Motion Field = Real world 3D motion
- Optical Flow Field = Projection of the motion field onto the 2d image (The apparent motion in the image due to the motion field)



Motion field and parallax

- $\mathbf{P}(t)$ is a moving 3D point
- Velocity of scene point: $\mathbf{V} = d\mathbf{P}/dt$
- $\mathbf{p}(t) = (x(t), y(t))$ is the projection of $\mathbf{P}$ in the image
- Apparent velocity $\mathbf{v}$ in the image: given by components $v_x = dx/dt$ and $v_y = dy/dt$
- These components are known as the *motion field* of the image

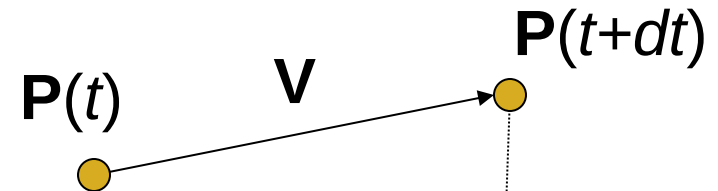


Motion field and parallax

Quotient rule:
 $D(f/g) = (g f' - g' f)/g^2$

$$\mathbf{V} = (V_x, V_y, V_z)$$

$$\mathbf{p} = f \frac{\mathbf{P}}{Z}$$



To find image velocity $\mathbf{v}$, differentiate $\mathbf{p}$ with respect to t (using quotient rule):

$$\mathbf{v} = f \frac{Z\mathbf{V} - V_z\mathbf{P}}{Z^2}$$

$$v_x = \frac{fV_x - V_zx}{Z} \quad v_y = \frac{fV_y - V_zy}{Z}$$

$$v_z = \frac{fV_z - V_zf}{Z} = 0$$

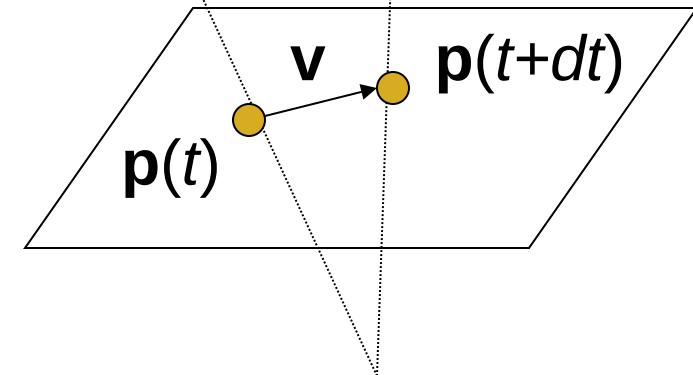


Image motion is a function of both the 3D motion ($\mathbf{V}$) and the depth of the 3D point (Z)

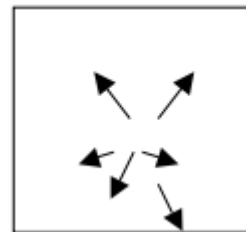
Motion field and parallax

- Pure translation: $\mathbf{V}$ is constant everywhere

$$\mathbf{v} = \frac{1}{Z}(\mathbf{v}_0 - V_z \mathbf{p}),$$

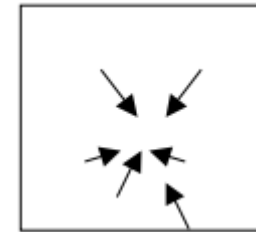
$$\mathbf{v}_0 = (fV_x, fV_y)$$

- V_z is nonzero:
 - Every motion vector points toward (or away from) $\mathbf{v}_0$, the vanishing point of the translation direction



$$V_z < 0$$

camera moving
towards object



$$V_z > 0$$

camera moving
away from object

Motion field and parallax

- Pure translation: $\mathbf{V}$ is constant everywhere

$$v_x = \frac{fV_x - V_z x}{Z} \quad \mathbf{v} = \frac{1}{Z} (\mathbf{v}_0 - V_z \mathbf{p}),$$

$$v_y = \frac{fV_y - V_z y}{Z} \quad \mathbf{v}_0 = (fV_x, fV_y)$$

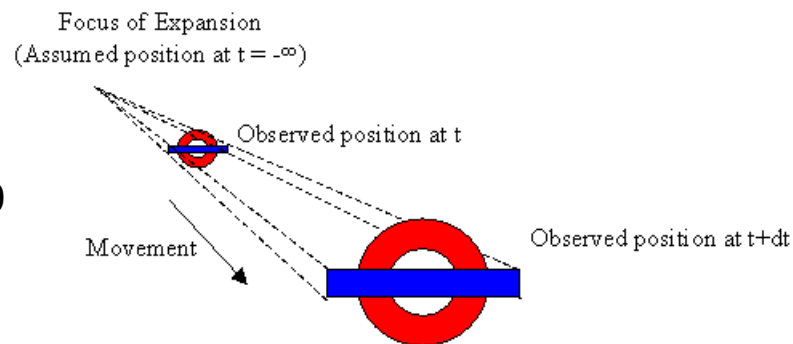
Define:

$$p_0 = \begin{bmatrix} x_0 \\ y_0 \\ f \end{bmatrix} = \begin{bmatrix} f \frac{V_x}{V_z} \\ f \frac{V_y}{V_z} \\ f \end{bmatrix}$$

$$x_t = f \frac{X + V_x t}{Z + V_z t}$$

$$t \rightarrow \infty$$

$$x_t = f \frac{V_x}{V_z} = x_0$$



focus of expansion/contraction

p_0 is the vanishing point of the direction of translation

p_0 is the intersection of the ray parallel to the translation vector and the image plane

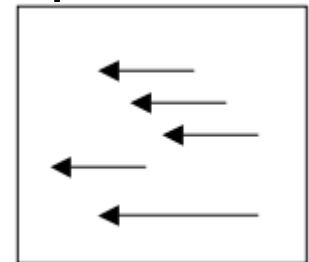
Motion field and parallax

- Pure translation: $\mathbf{V}$ is constant everywhere

$$\mathbf{v} = \frac{1}{Z}(\mathbf{v}_0 - V_z \mathbf{p}),$$

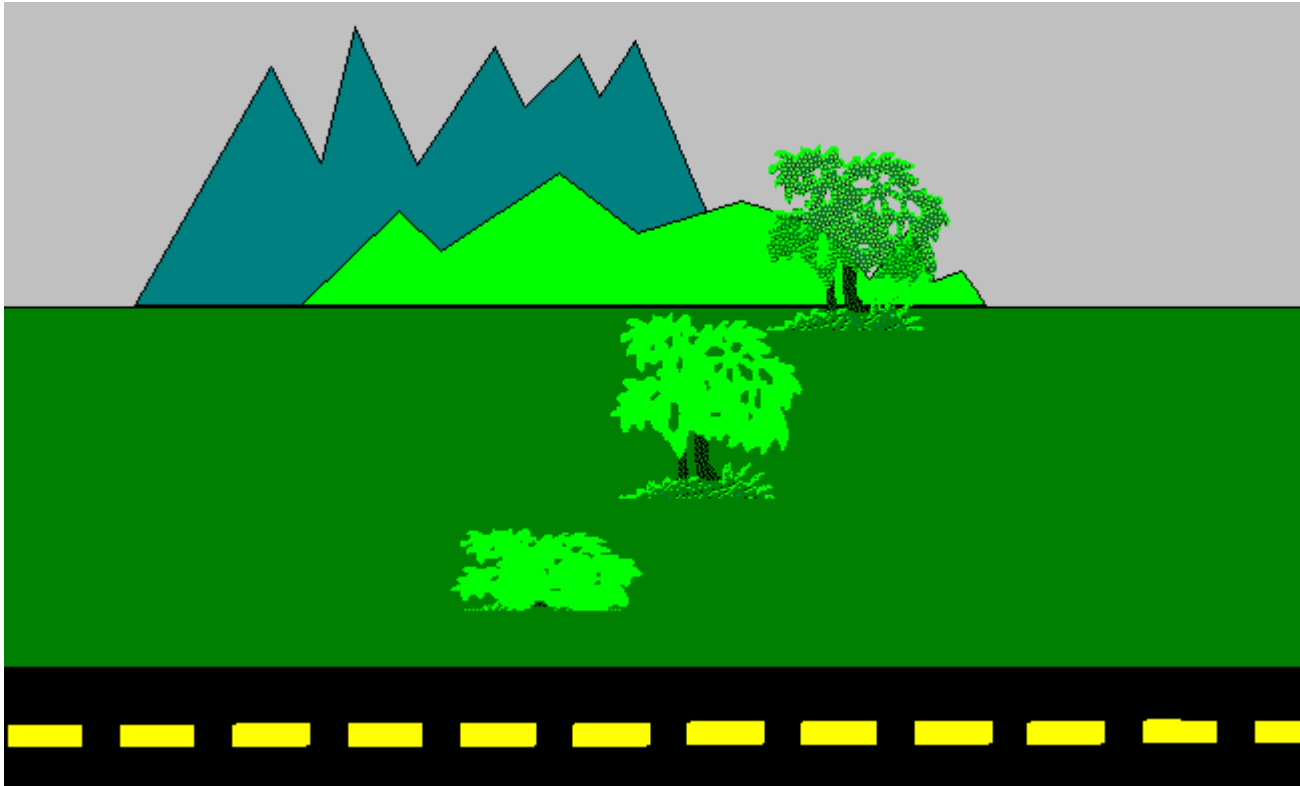
$$\mathbf{v}_0 = (fV_x, fV_y)$$

- V_z is nonzero:
 - Every motion vector points toward (or away from) $\mathbf{v}_0$, the vanishing point of the translation direction
- V_z is zero:
 - Motion is parallel to the image plane, all the motion vectors are parallel
- The length of the motion vectors is inversely proportional to the depth Z



Motion parallax

Object that are farther away move slowly across the retina as we move by them.

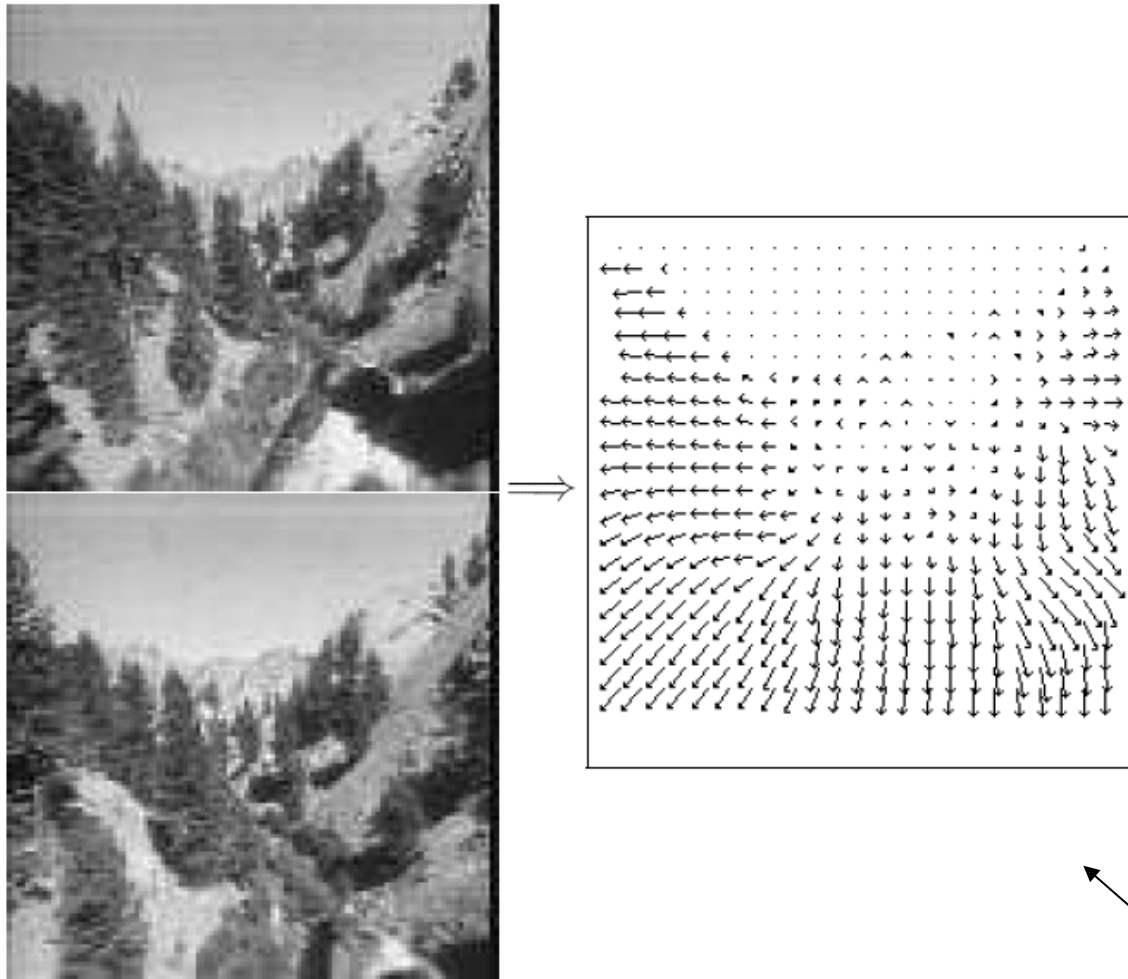


Motion parallax

Object that are farther away move slowly across the retina as we move by them.



Motion field + camera motion



Length of flow vectors inversely proportional to depth Z of 3d point

Figure 1.2: Two images taken from a helicopter flying through a canyon and the computed optical flow field.

points closer to the camera move more quickly across the image plane

Figure from Michael Black, Ph.D. Thesis

General Visual Motion

Let us assume there is one rigid object and the camera moves with translational velocity $\mathbf{T}$ and rotational velocity $\boldsymbol{\omega}$

For a given point $\mathbf{P} = [X, Y, Z]^T$ on the object, we have

$$\mathbf{v} = f \frac{Z\mathbf{V} - V_z\mathbf{P}}{Z^2}$$

where

$$\mathbf{V} = -\mathbf{T} - \boldsymbol{\omega} \times \mathbf{P}, \quad V_z = -T_z - \omega_x Y + \omega_y X$$

The Result:

$$\frac{du}{dt} = (T_z u - f T_x)/Z - f\omega_y + \omega_z v + \omega_x uv/f - \omega_y u^2/f$$

$$\frac{dv}{dt} = (T_z v - f T_y)/Z + f\omega_x - \omega_z u - \omega_y uv/f + \omega_x v^2/f$$

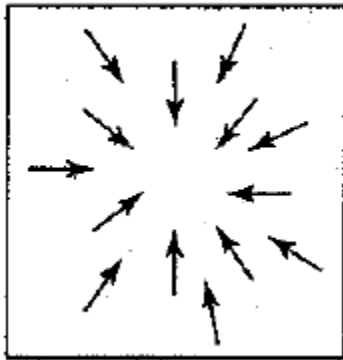


Motion due to translation:
depends on depth

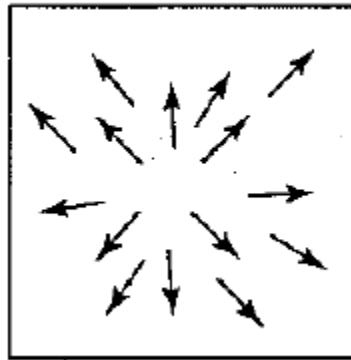


Motion due to rotation:
independent of depth

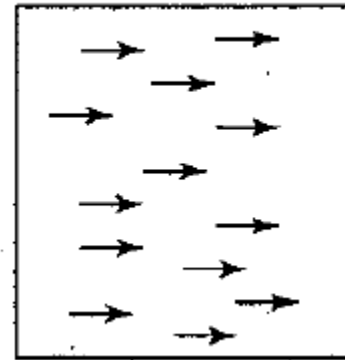
Motion field + camera motion



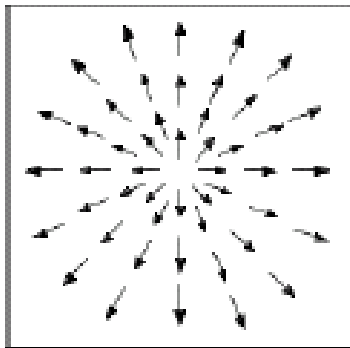
Zoom out



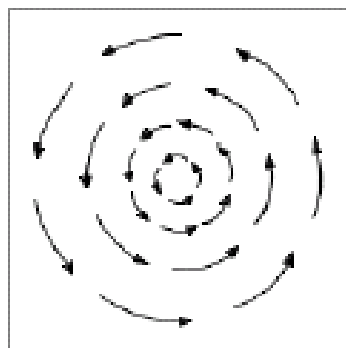
Zoom in



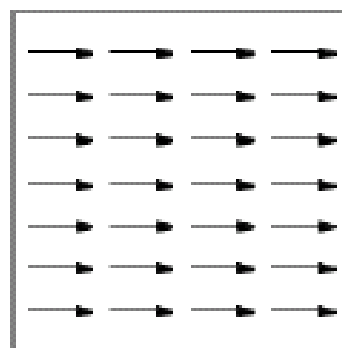
Pan right to left



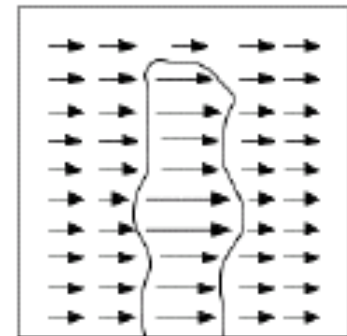
Forward motion



Rotation



Horizontal translation



Closer objects appear to move faster!!

Motion estimation techniques

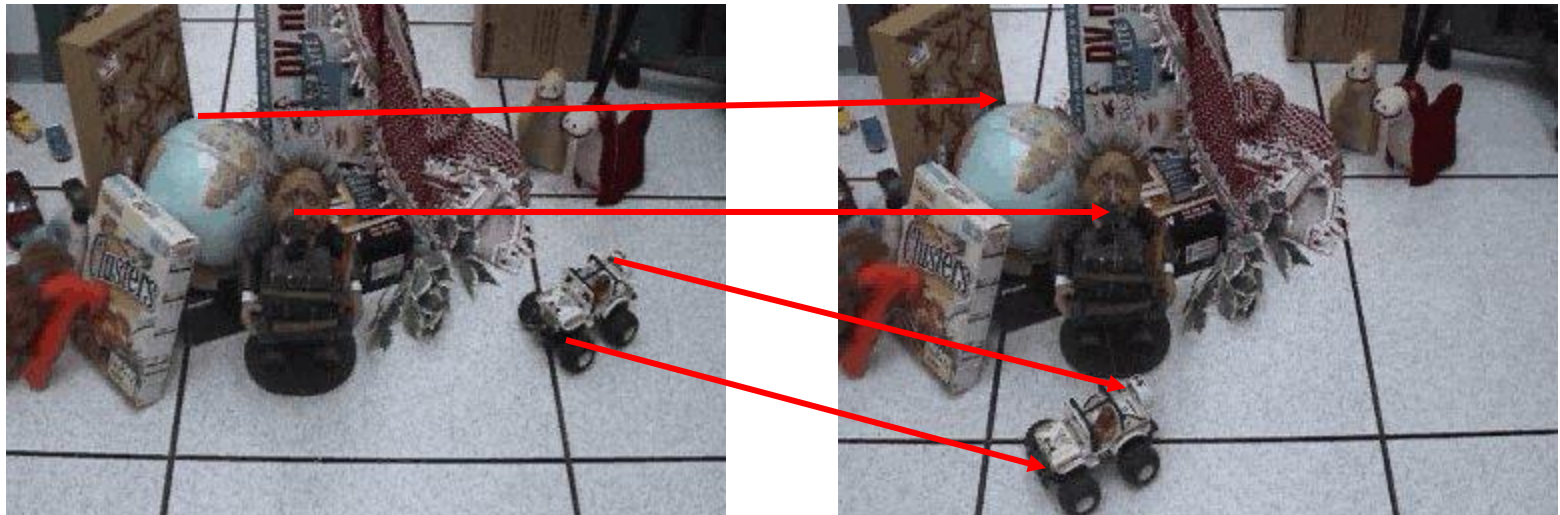
- Feature-based methods
 - Extract visual features (corners, textured areas) and track them over multiple frames
 - Sparse motion fields, but more robust tracking
 - Suitable when image motion is large (10s of pixels)
- Direct methods
 - Directly recover image motion at each pixel from spatio-temporal image brightness variations
 - Dense motion fields, but sensitive to appearance variations
 - Suitable for video and when image motion is small

Motion estimation techniques

- Feature-based methods
 - Extract visual features (corners, textured areas) and track them over multiple frames
 - Sparse motion fields, but more robust tracking
 - Suitable when image motion is large (10s of pixels)
- Direct methods
 - Directly recover image motion at each pixel from spatio-temporal image brightness variations
 - Dense motion fields, but sensitive to appearance variations
 - Suitable for video and when image motion is small

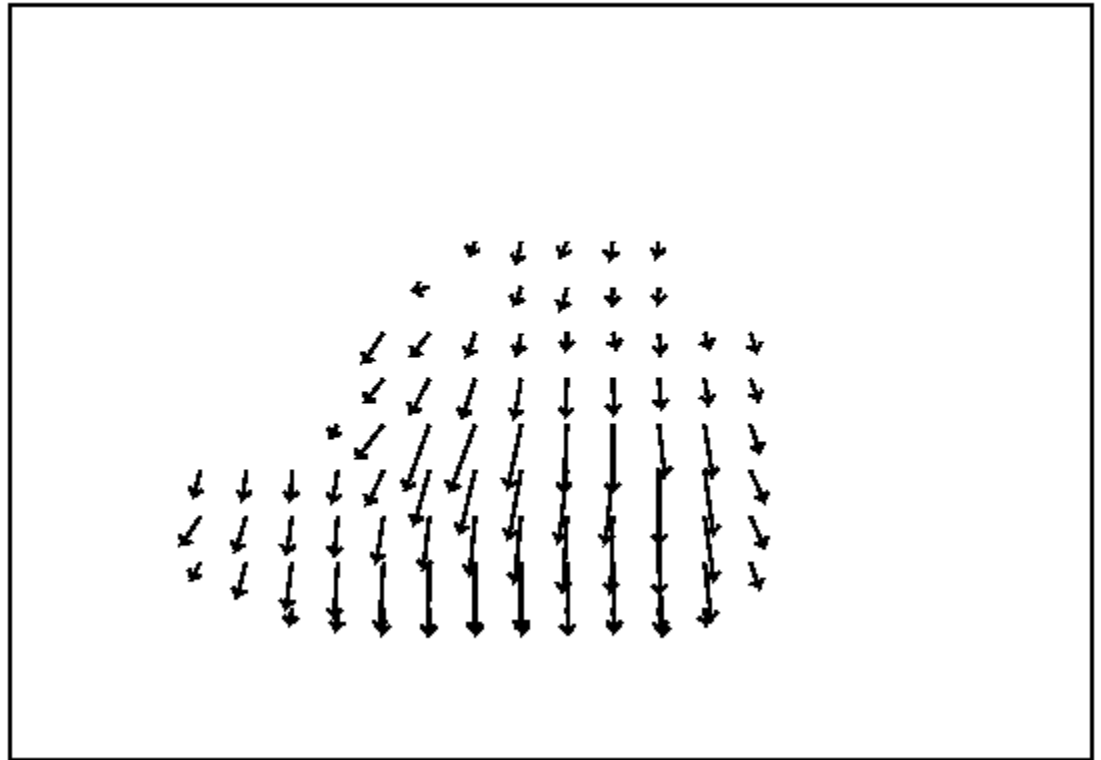
Optical flow

- Definition: optical flow is the *apparent* motion of brightness patterns in the image
- Ideally, optical flow would be the same as the motion field
- Have to be careful: apparent motion can be caused by lighting changes without any actual motion



Where did each pixel in image 1 go to in image 2

Optical Flow

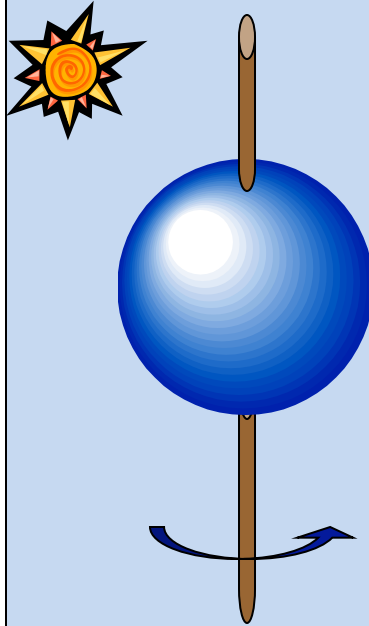


Pierre Kornprobst's Demo

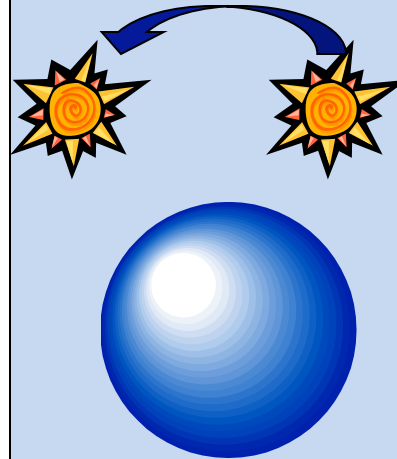
When does it break?



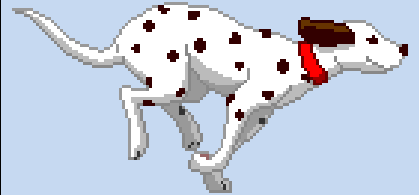
The screen is stationary yet displays motion



Homogeneous objects generate zero optical flow.



Fixed sphere. Changing light source.



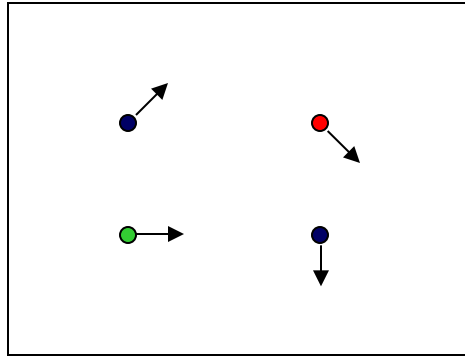
Non-rigid texture motion

The Optical Flow Field

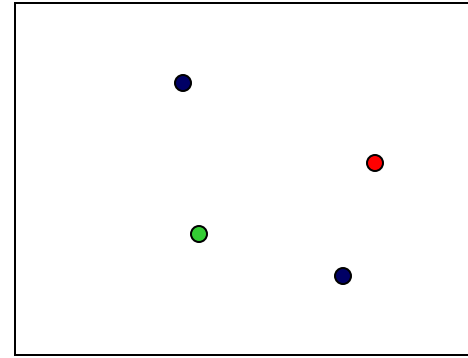
Still, in many cases it does work....

- Goal:
Find for each pixel a velocity vector $\vec{u} = (u, v)$
which says:
 - How quickly is the pixel moving across the image
 - In which direction it is moving

Estimating optical flow



$I(x,y,t-1)$



$I(x,y,t)$

- Given two subsequent frames, estimate the apparent motion field between them.
- Key assumptions
 - **Brightness constancy:** projection of the same point looks the same in every frame
 - **Small motion:** points do not move very far
 - **Spatial coherence:** points move like their neighbors

Brightness constancy

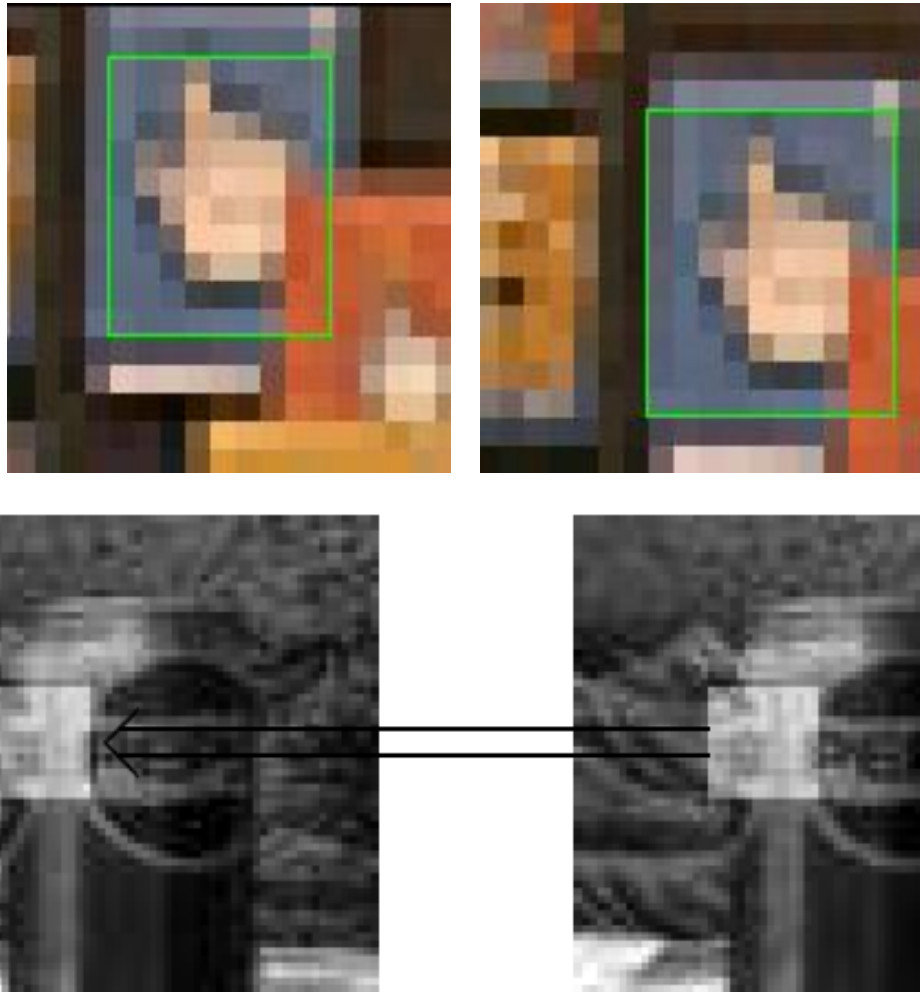
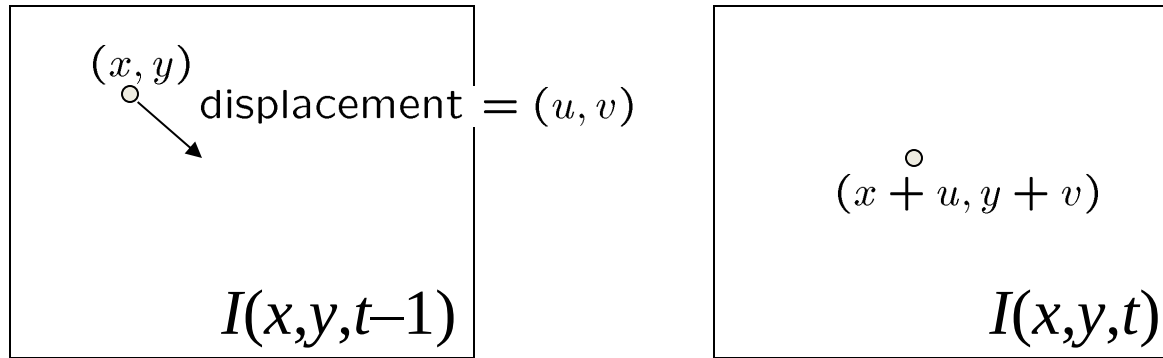


Figure 1.5: Data conservation assumption. The highlighted region in the right image looks roughly the same as the region in the left image, despite the fact that it has moved.

Figure by Michael Black

The brightness constancy constraint



- Brightness Constancy Equation:

$$I(x, y, t - 1) = I(x + u(x, y), y + v(x, y), t)$$

Can be written as:

shorthand: $I_x = \frac{\partial I}{\partial x}$

$$I(x, y, t - 1) \approx I(x, y, t) + I_x \cdot u(x, y) + I_y \cdot v(x, y)$$

So, $I_x \cdot u + I_y \cdot v + I_t \approx 0$

The brightness constancy constraint

$$I_x \cdot u + I_y \cdot v + I_t = 0$$

- How many equations and unknowns per pixel?
 - One equation, two unknowns

- Intuitively, what does this constraint mean?

$$\nabla I \cdot (u, v) + I_t = 0$$

- The component of the flow perpendicular to the gradient (i.e., parallel to the edge) is unknown

The brightness constancy constraint

$$I_x \cdot u + I_y \cdot v + I_t = 0$$

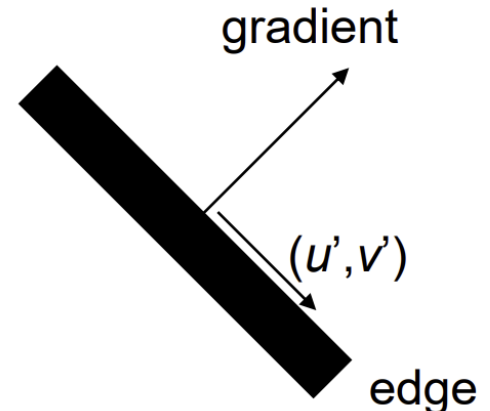
- How many equations and unknowns per pixel?
 - One equation, two unknowns

- Intuitively, what does this constraint mean?

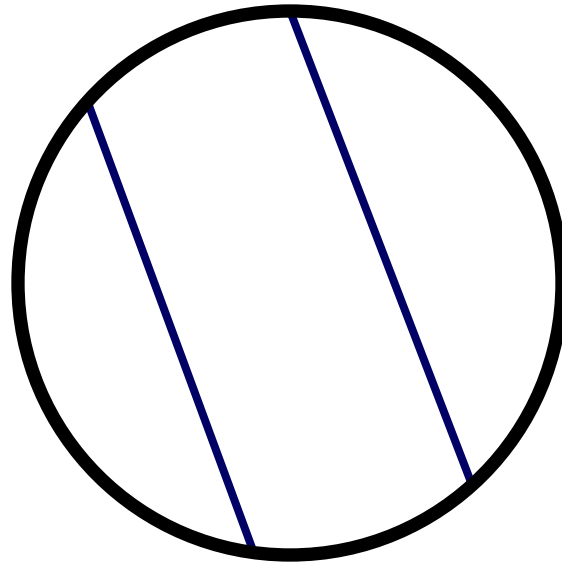
$$\nabla I \cdot (u, v) + I_t = 0$$

- The component of the flow perpendicular to the gradient (i.e., parallel to the edge) is unknown

If (u, v) satisfies the equation,
so does $(u+u', v+v')$ if $\nabla I \cdot (u', v') = 0$

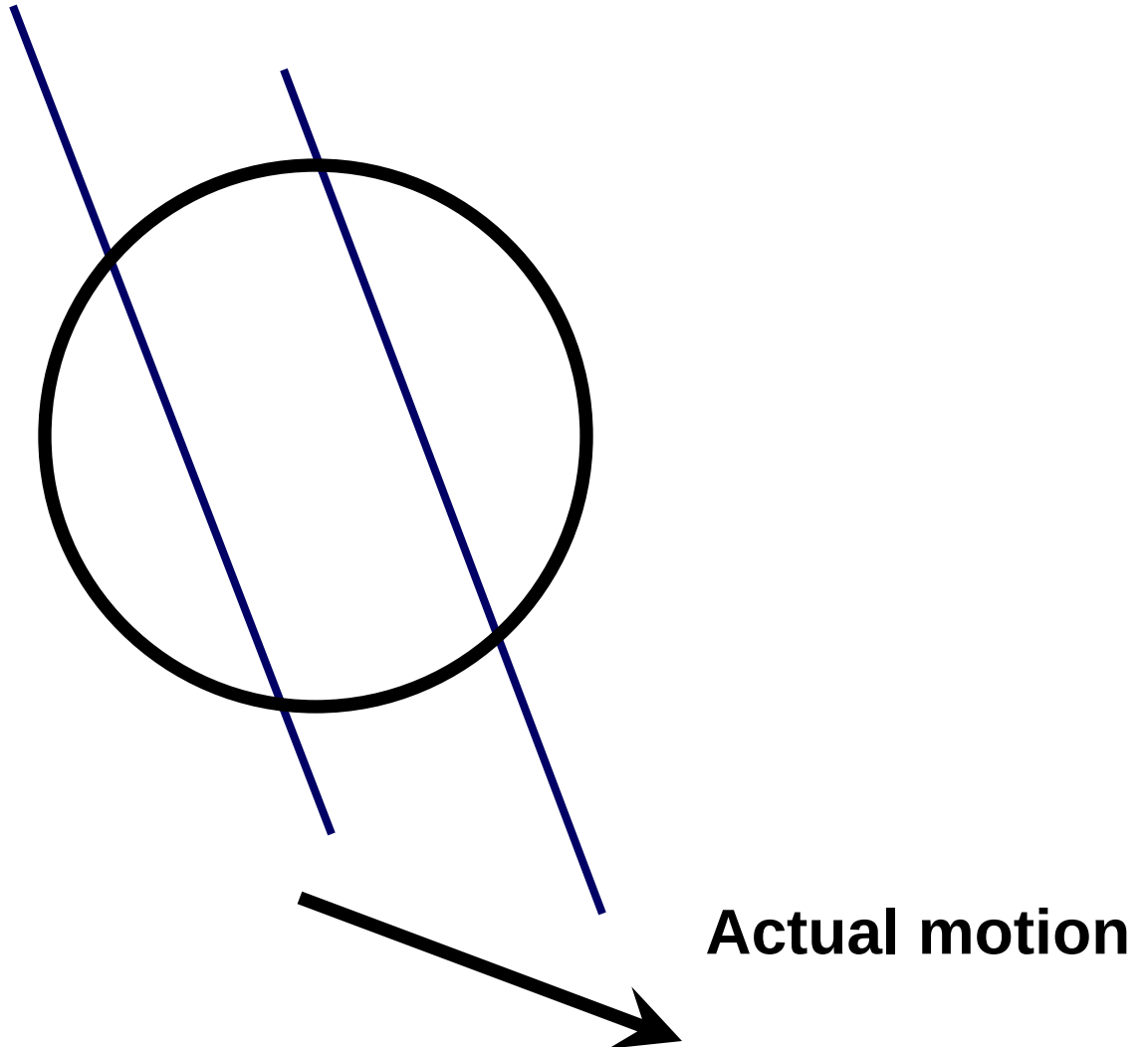


The aperture problem

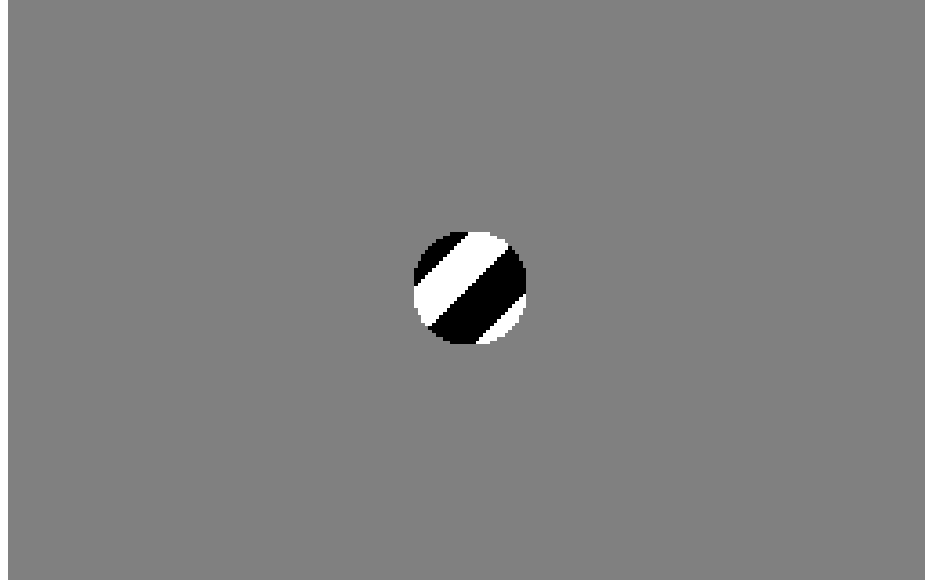


Perceived motion

The aperture problem

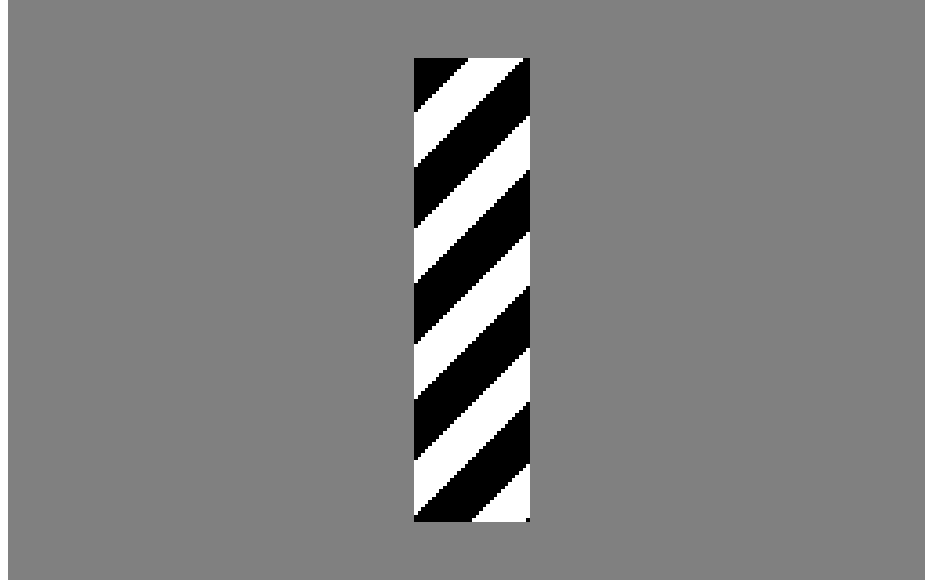


The barber pole illusion



http://en.wikipedia.org/wiki/Barberpole_illusion

The barber pole illusion



http://en.wikipedia.org/wiki/Barberpole_illusion

The barber pole illusion



http://en.wikipedia.org/wiki/Barberpole_illusion

Lucas-Kanade: Solving the aperture problem (grayscale image)

- How to get more equations for a pixel?
- **Spatial coherence constraint:** pretend the pixel's neighbors have the same (u,v)
 - If we use a 5x5 window, that gives us 25 equations per pixel

$$0 = I_t(\mathbf{p}_i) + \nabla I(\mathbf{p}_i) \cdot [u \ v]$$

$$\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ I_x(\mathbf{p}_2) & I_y(\mathbf{p}_2) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{25}) & I_y(\mathbf{p}_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\mathbf{p}_1) \\ I_t(\mathbf{p}_2) \\ \vdots \\ I_t(\mathbf{p}_{25}) \end{bmatrix}$$

$$\begin{matrix} A & d & = & b \\ 25 \times 2 & 2 \times 1 & & 25 \times 1 \end{matrix}$$

Lucas-Kanade: Solving the aperture problem

Prob: we have more equations than unknowns

$$\underset{25 \times 2}{A} \underset{2 \times 1}{d} = \underset{25 \times 1}{b} \longrightarrow \text{minimize } \|Ad - b\|^2$$

Solution: solve least squares problem

- minimum least squares solution given by solution (in d) of:

$$\underset{2 \times 2}{(A^T A)} \underset{2 \times 1}{d} = \underset{2 \times 1}{A^T b}$$

$$\underset{A^T A}{\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix}} \begin{bmatrix} u \\ v \end{bmatrix} = - \underset{A^T b}{\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}}$$

- The summations are over all pixels in the K x K window
- This technique was first proposed by Lucas & Kanade (1981)

Conditions for solvability

Look Familiar?

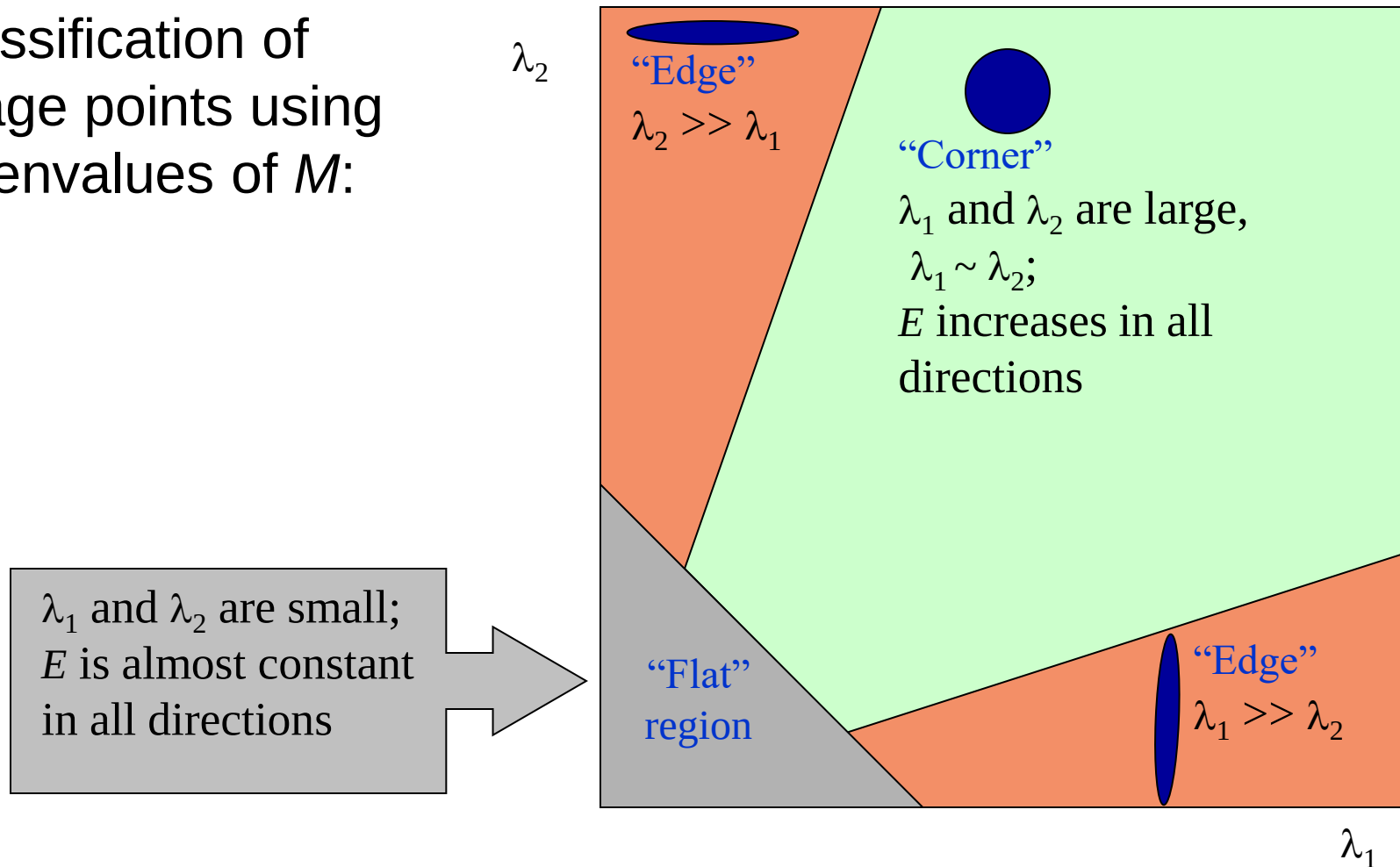
$$\underbrace{\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix}}_{A^T A} \begin{bmatrix} u \\ v \end{bmatrix} = - \underbrace{\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}}_{A^T b}$$

When is this solvable?

- $A^T A$ should be invertible
- $A^T A$ should not be too small
 - eigenvalues λ_1 and λ_2 of $A^T A$ should not be too small
- $A^T A$ should be well-conditioned
 - λ_1 / λ_2 should not be too large ($\lambda_1 =$ larger eigenvalue)

Conditions for solvability

Classification of image points using eigenvalues of M :



Edge



- gradients very large or very small
- large λ_1 , small λ_2

Low-texture region



- gradients have small magnitude
- small λ_1 , small λ_2

High-texture region



- gradients are different, large magnitudes
- large λ_1 , large λ_2

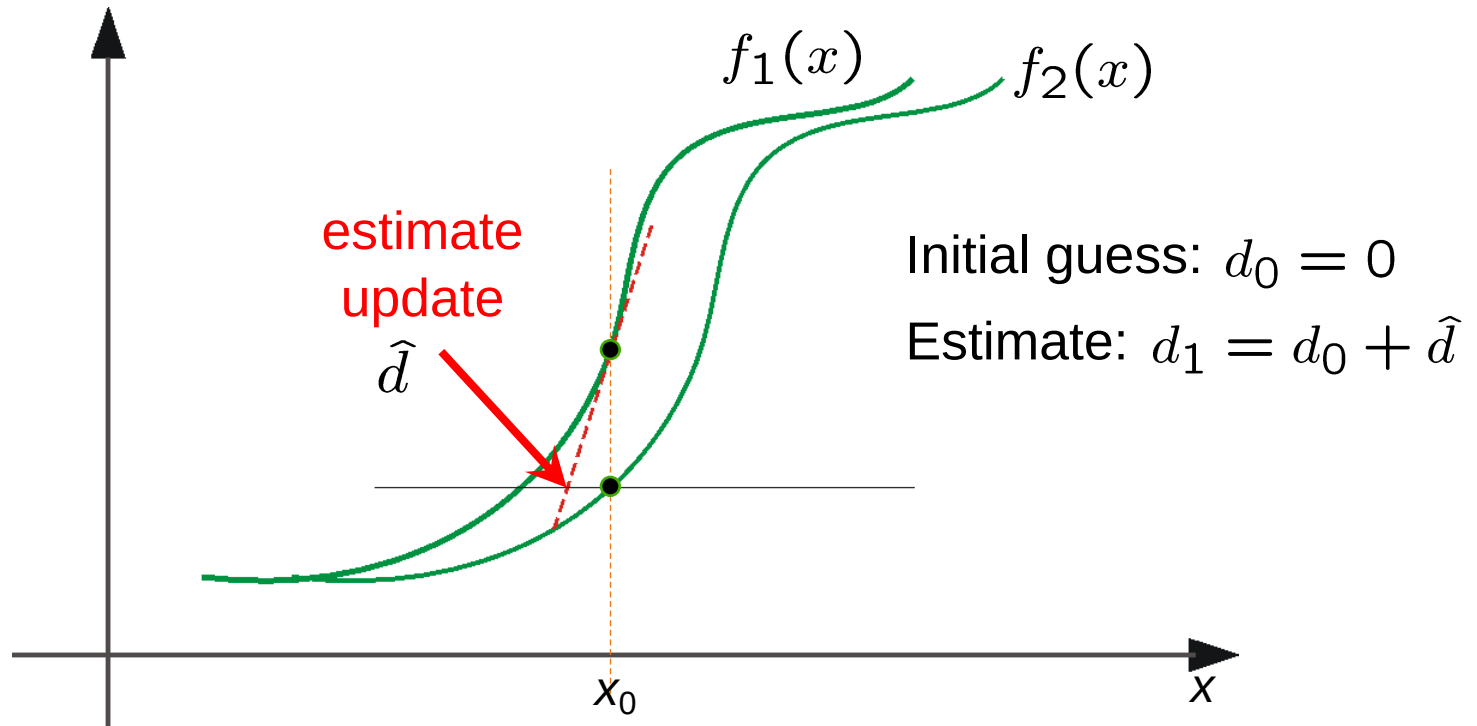
Can we measure optical flow reliability?

- Can we measure “quality” of optical flow in regions from just a single image?
- High Quality / Good features to track:
 - - Harris corners (guarantee small error sensitivity)
- Poor Quality / Bad features to track:
 - - Image points when either λ_1 or λ_2 (or both) is small (i.e., edges or uniform textured regions)

Iterative Refinement

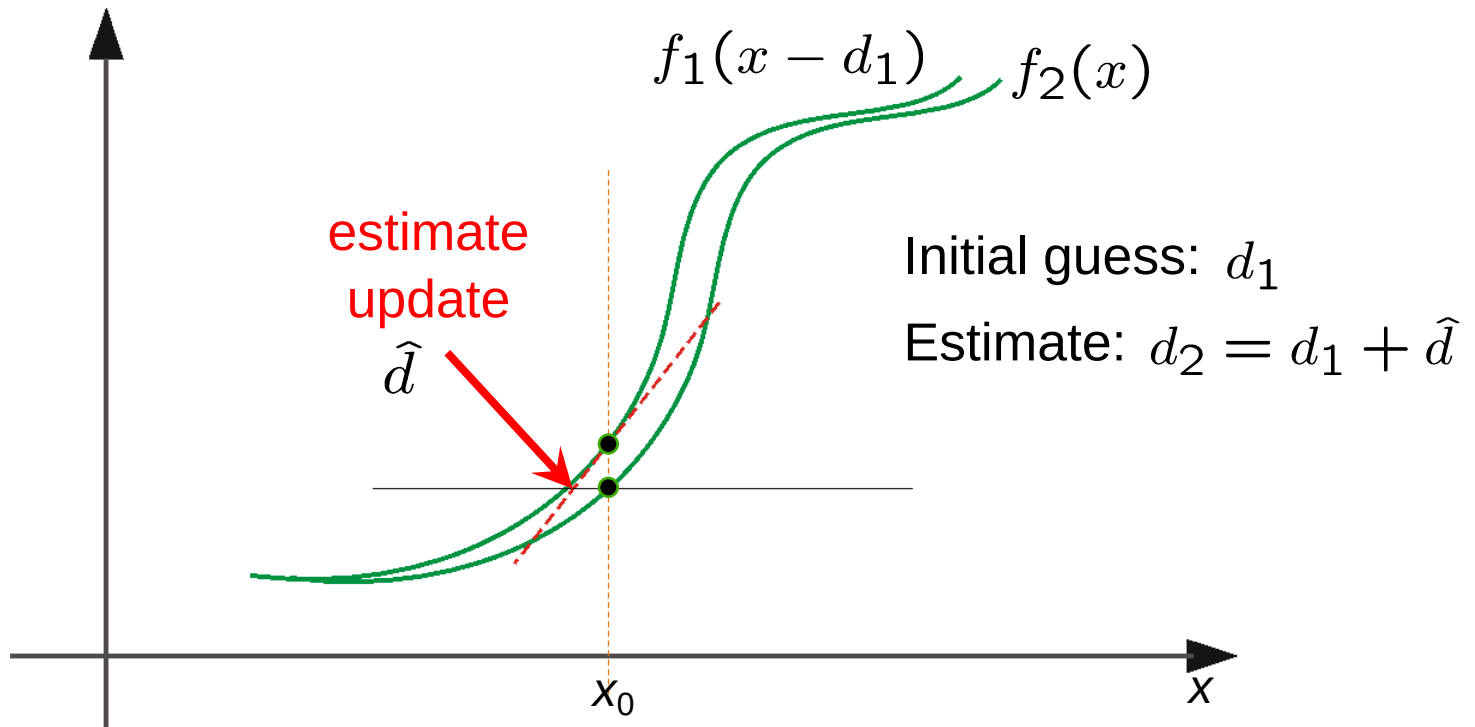
- Estimate velocity at each pixel using one iteration of Lucas and Kanade estimation
- Warp one image toward the other using the estimated flow field
(easier said than done)
- Refine estimate by repeating the process

Optical Flow: Iterative Estimation

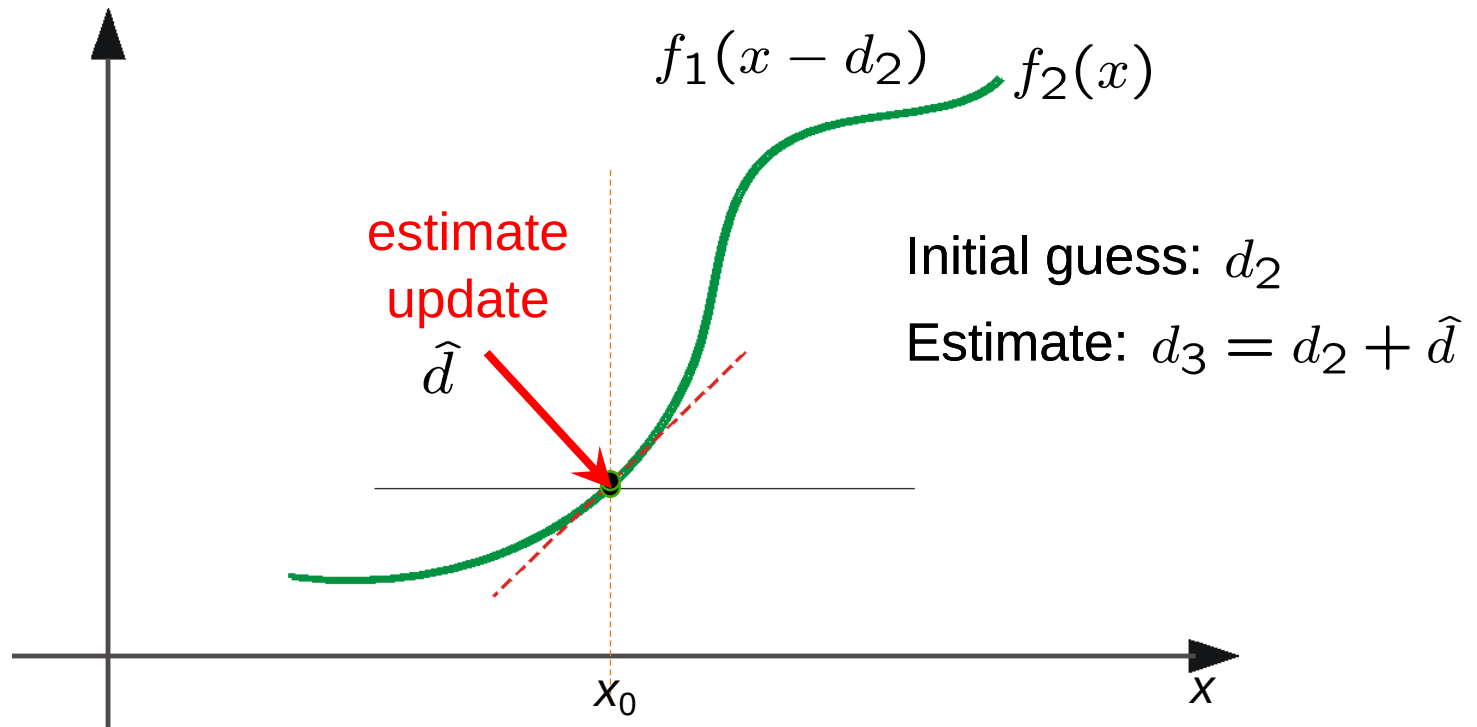


(using d for *displacement* here instead of u)

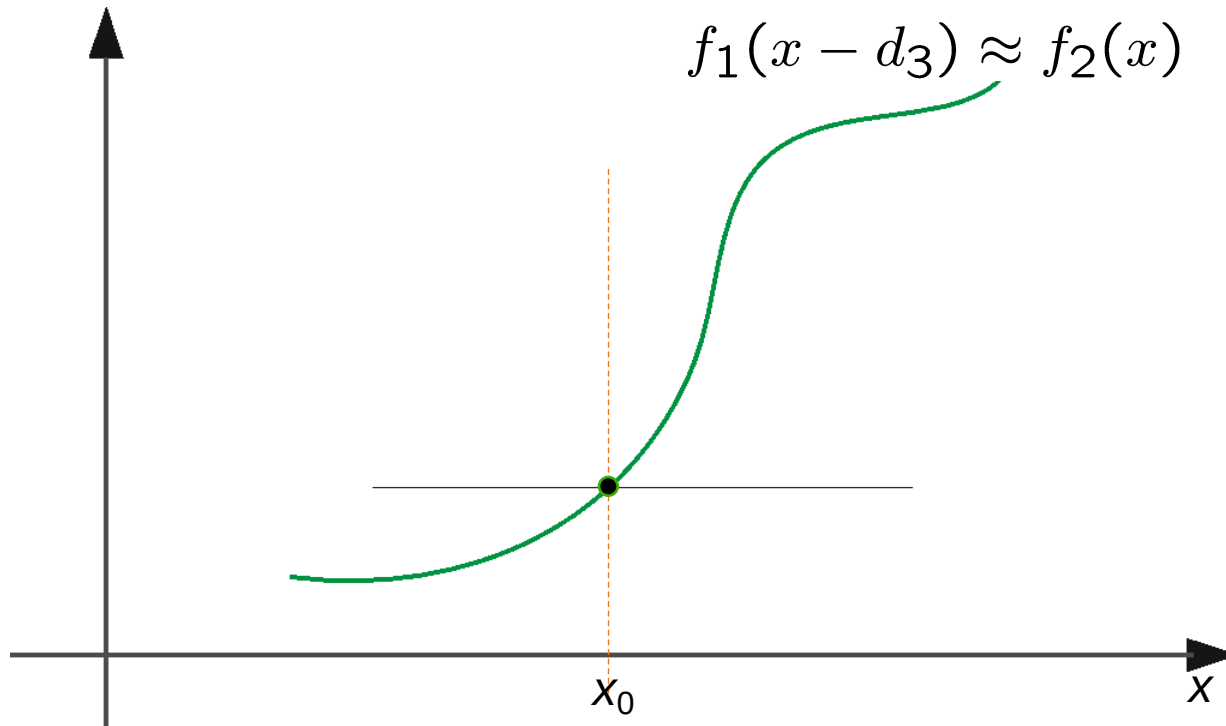
Optical Flow: Iterative Estimation



Optical Flow: Iterative Estimation



Optical Flow: Iterative Estimation



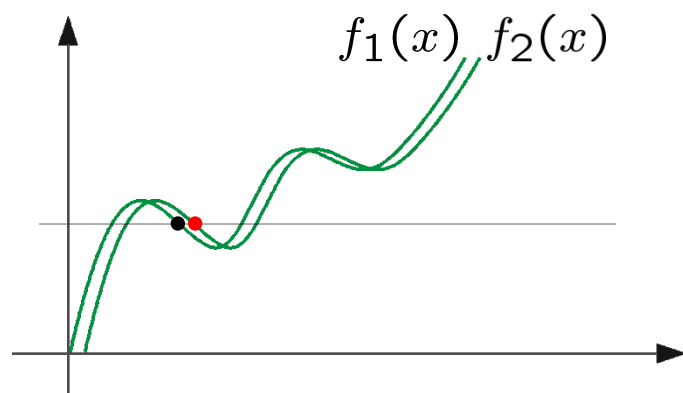
Optical Flow: Iterative Estimation

- Some Implementation Issues:
 - Warping is not easy (ensure that errors in warping are smaller than the estimate refinement)
 - Warp one image, take derivatives of the other so you don't need to re-compute the gradient after each iteration.
 - Often useful to low-pass filter the images before motion estimation (for better derivative estimation, and linear approximations to image intensity)

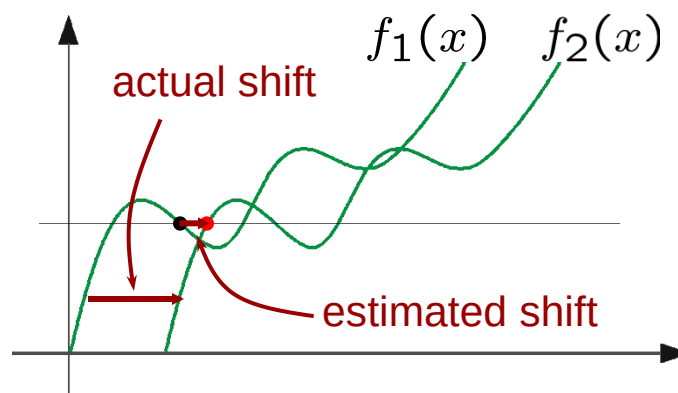
Optical Flow: Aliasing

Temporal aliasing causes ambiguities in optical flow because images can have many pixels with the same intensity.

I.e., how do we know which 'correspondence' is correct?



*nearest match is correct
(no aliasing)*



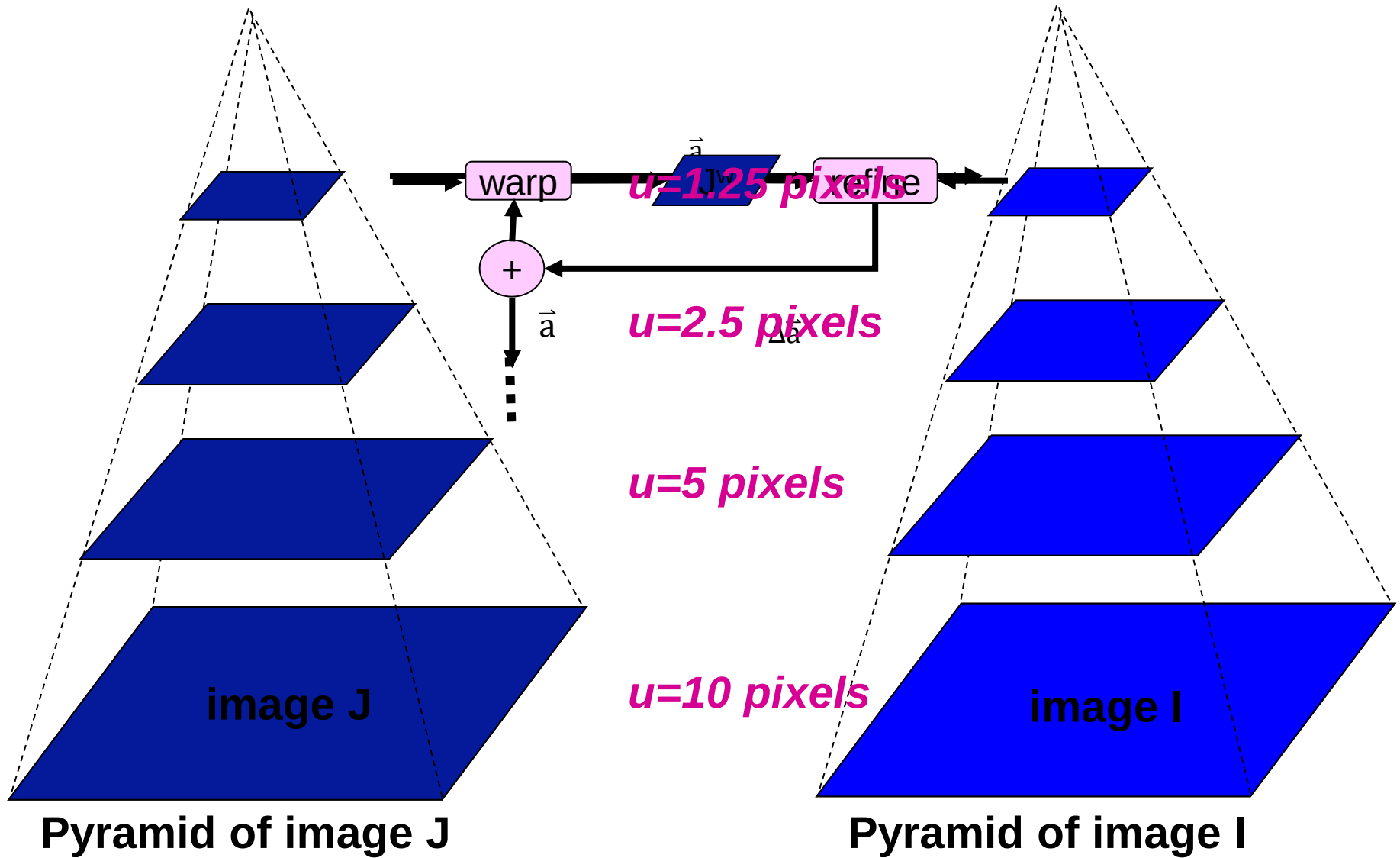
*nearest match is incorrect
(aliasing)*

To overcome aliasing: coarse-to-fine estimation.

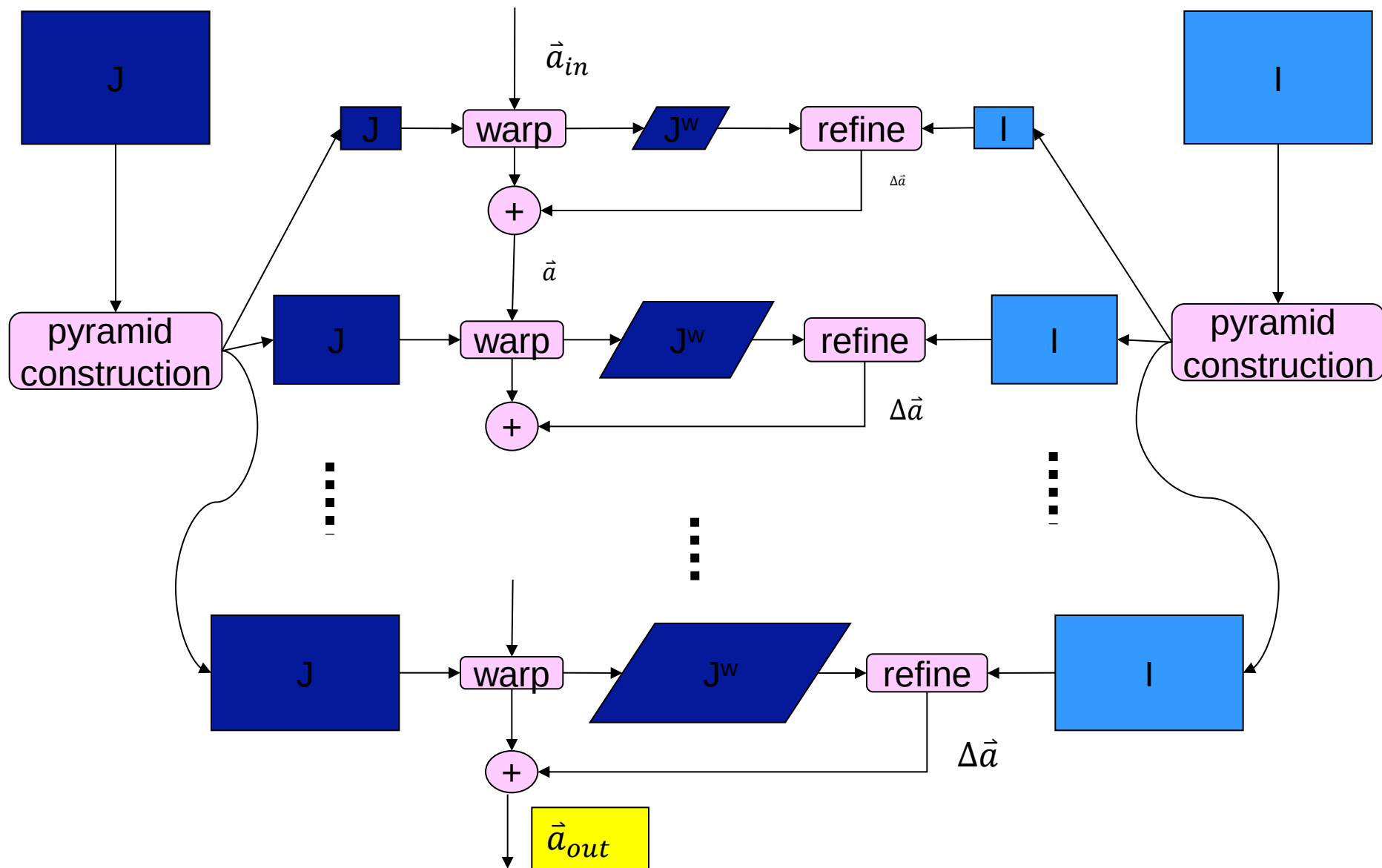
Limits of the gradient method

- Fails when intensity structure in window is poor
- Fails when the displacement is large (typical operating range is motion of 1 pixel)
- Also, brightness is not strictly constant in images
 - *actually less problematic than it appears, since we can pre-filter images to make them look similar*

Coarse-to-Fine Estimation

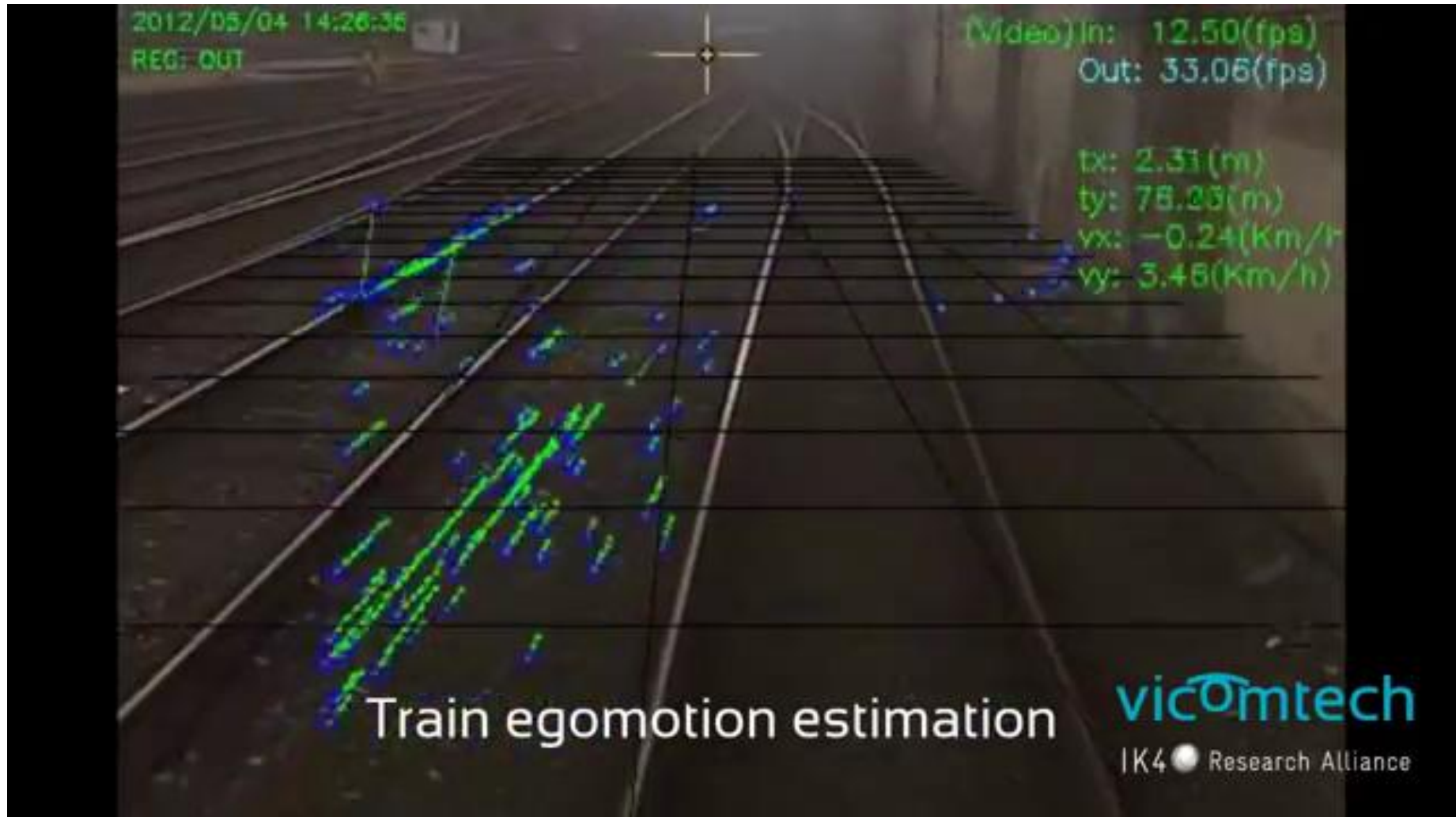


Coarse-to-Fine Estimation



Applications of Optical Flow

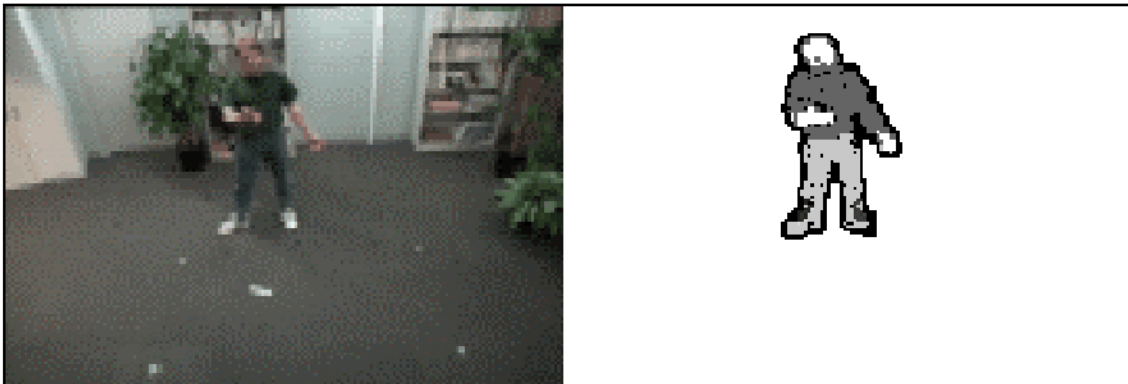
Egomotion Estimation on the Railway



Applications to Segmentation

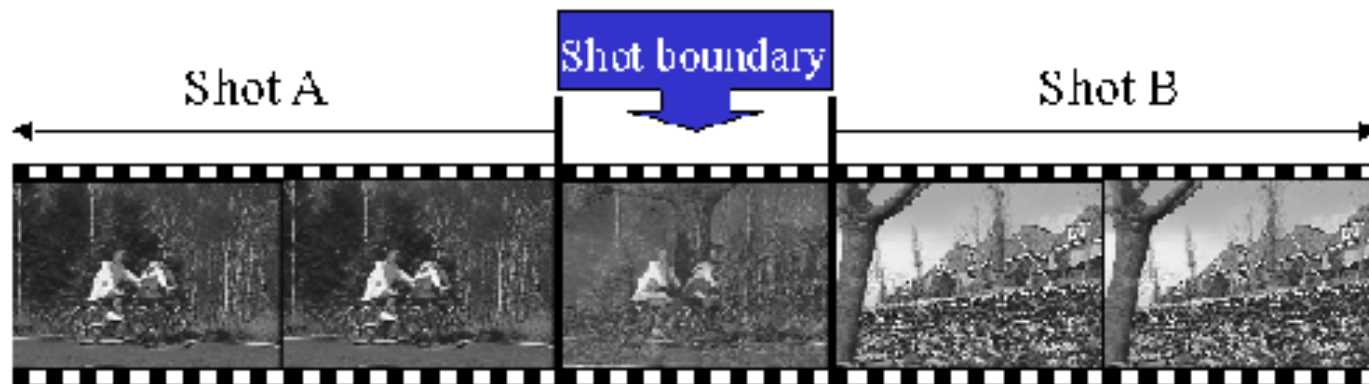
- Background subtraction
 - A static camera is observing a scene
 - Goal: separate the static *background* from the moving *foreground*

How to come up with background frame estimate without access to “empty” scene?



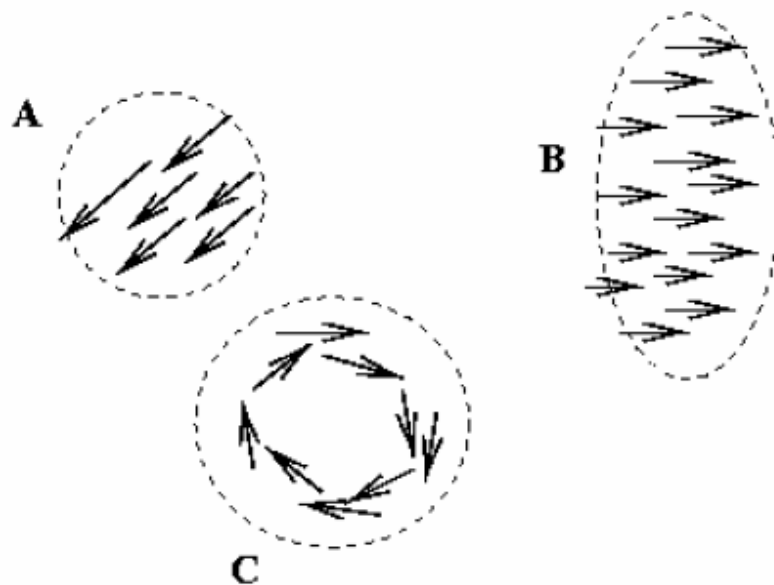
Applications to Segmentation

- Background subtraction
- Shot boundary detection
 - Commercial video is usually composed of *shots* or sequences showing the same objects or scene
 - Goal: segment video into shots for summarization and browsing (each shot can be represented by a single keyframe in a user interface)
 - Difference from background subtraction: the camera is not necessarily stationary



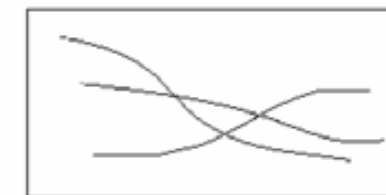
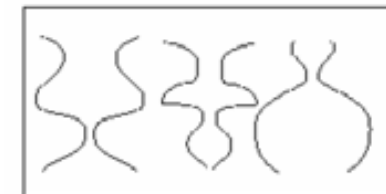
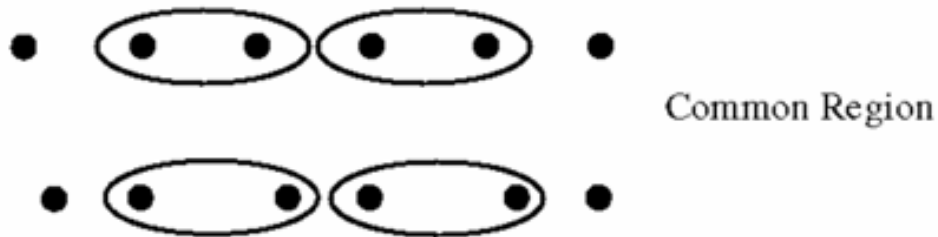
Applications To Segmentation

- Background subtraction
- Shot boundary detection
- Motion segmentation
 - Segment the video into multiple *coherently* moving objects



Motion and perceptual organization

- Sometimes, motion is the only cue



Parallelism

Symmetry

Continuity

Closure

Motion and perceptual organization

- Sometimes, motion is foremost cue



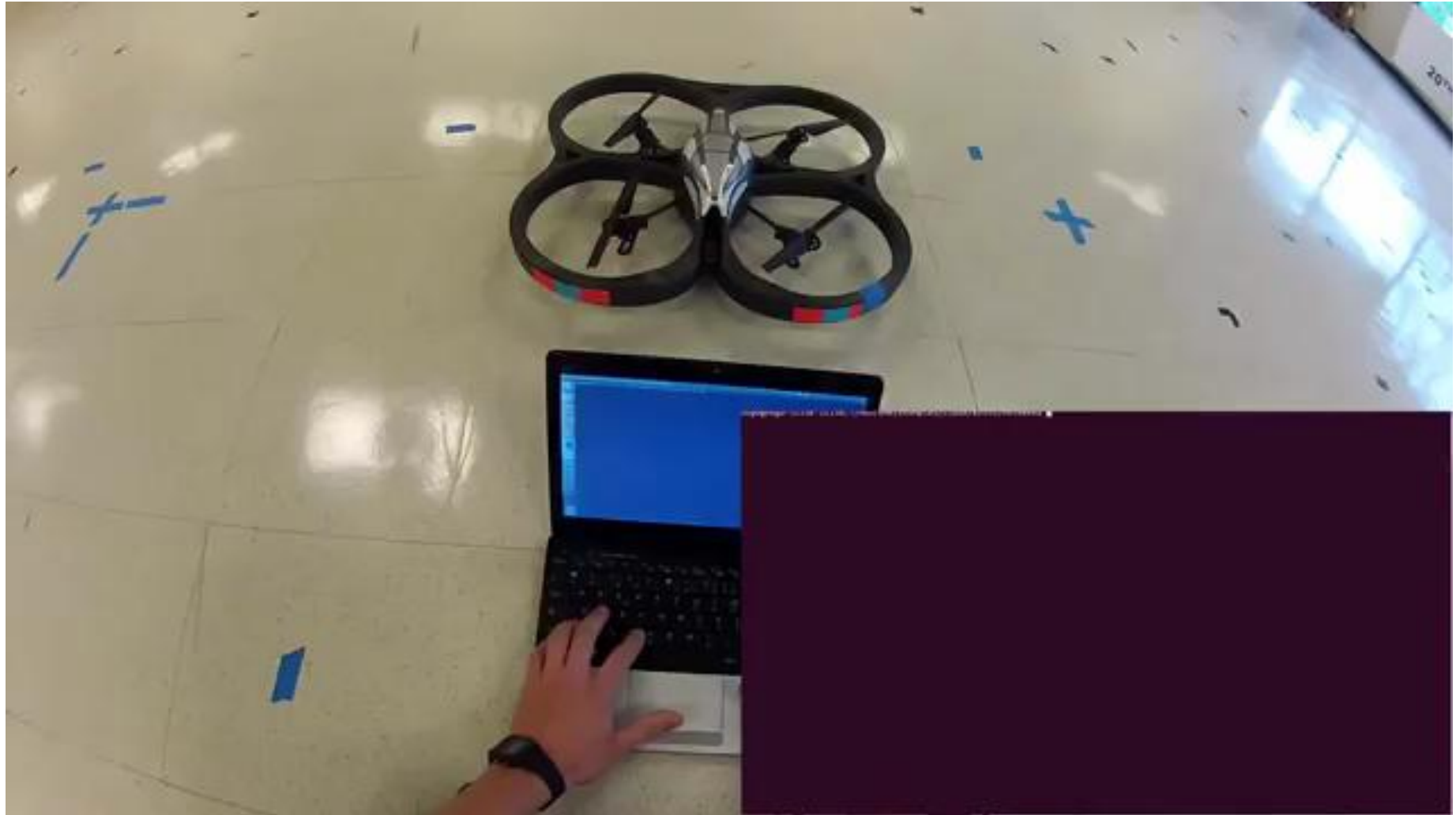
Motion and perceptual organization

- Even “impoverished” motion data can evoke a strong percept



Sources: Maas 1971 with Johansson; downloaded from Youtube.

Aerial Vehicle Target Tracking



A Camera Mouse

- Video interface: use feature tracking as mouse replacement



- User clicks on the feature to be tracked
- Take the 15x15 pixel square of the feature
- In the next image do a search to find the 15x15 region with the highest correlation
- Move the mouse pointer accordingly
- Repeat in the background every 1/30th of a second

A Camera Mouse

- Specialized software for communication, games



James Gips and Margrit Betke
<http://www.bc.edu/schools/csom/eagleeyes/>

Optical Flow for Games!

Simple Optical Flow Application

Motion Paint: an example use of optical flow

Use optical flow to track brush strokes, in order to animate them to follow underlying scene motion.



<http://www.fxguide.com/article333.html>

Motion Paint: an example use of optical flow

